



Chapter 1

Digital Systems and Binary Numbers

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

本教學投影片係屬教科書著作之延伸，亦受著作權法之保護。

請依出版社授權使用規範利用本教學資源，非經授權許可不得以影印、拷貝、列印等方法進行重製，亦不得改作、公開展示著作內容，亦請勿對第三人公開傳輸、散布。

Chapter Objectives

- Understand the binary number system.
- Know how to convert between binary, octal, decimal, and hexadecimal numbers.
- Know how to take the complement and reduced radix complement of a number.
- Know how to form the code of a number.
- Know how to form the parity bit of a word.

1.1 Digital Systems (1/2)

- What is signal?
 - An information variable represented by physical quantity
 - For digital systems, the variable takes on discrete values
 - Two levels, or binary values are the most prevalent values
 - Binary values are represented abstractly by:
 - digits 0 and 1
 - words (symbols) False (F) and True (T)
 - words (symbols) Low (L) and High (H)
 - words On and Off
 - Binary values are represented by values or ranges of values of physical quantities

1.1 Digital System (2/2)

- A digital system is an interconnection of digital modules.
- To understand the operation of each digital module, it is necessary to have a basic knowledge of digital circuits and their logical function.
- A major trend in digital design methodology is the use of a hardware description language (HDL) to describe and simulate the functionality of a digital circuit.
- It is important that students become familiar with an HDL-based design methodology.

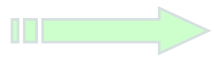
1.2 Binary Numbers (1/7)

- Decimal number $\dots a_5 a_4 a_3 a_2 a_1 \cdot a_{-1} a_{-2} a_{-3} \dots$



Decimal point

- The coefficients a_j are any of the 10 digits (0, 1, 2, ..., 9)
- The subscript value j gives the place value and the power of 10 by which the coefficient must be multiplied.



$$\dots + 10^5 a_5 + 10^4 a_4 + 10^3 a_3 + 10^2 a_2 + 10^1 a_1 + 10^0 a_0 + 10^{-1} a_{-1} + 10^{-2} a_{-2} + 10^{-3} a_{-3} + \dots$$

Example: $7,329 = 7 \times 10^3 + 3 \times 10^2 + 2 \times 10^1 + 9 \times 10^0$

- General form of base- r system

$$a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_2 \cdot r^2 + a_1 \cdot r^1 + a_0 + a_{-1} \cdot r^{-1} + a_{-2} \cdot r^{-2} + \dots + a_{-m} \cdot r^{-m}$$

Coefficient: $a_j = 0 \text{ to } r - 1$

1.2 Binary Numbers (2/7)

- Example: Base-2 number

$$(11010.11)_2 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = (26.75)_{10}$$

- Example: Base-5 number

$$(4021.2)_5 = 4 \times 5^3 + 0 \times 5^2 + 2 \times 5^1 + 1 \times 5^0 + 2 \times 5^{-1} = (511.5)_{10}$$

- Example: Base-8 number

$$(127.4)_8 = 1 \times 8^3 + 2 \times 8^2 + 7 \times 8^1 + 4 \times 8^0 = (87.5)_{10}$$

- Example: Base-16 number

$$(B65F)_{16} = 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0 = (46,687)_{10}$$

1.2 Binary Numbers (3/7)

- Example: Base-2 number

$$(110101)_2 = 32 + 16 + 4 + 1 = (53)_{10}$$

- Special Powers of 2 in computer science

- $2^{10} = 1,024$ is Kilo, denoted “K”
- $2^{20} = 1,048,576$ is Mega, denoted “M”
- $2^{30} = 1,073,741,824$ is Giga, denoted “G”
- 2^{40} is Tera, denoted “T”

- Example:

- $4K = 4 \times 2^{10} = 2^2 \times 2^{10} = 2^{12} = 4,096$
- $16M = 16 \times 2^{20} = 2^4 \times 2^{20} = 2^{24} = 16,777,216$

1.2 Binary Numbers (4/7)

Table 1.1
Powers of Two

n	2^n	n	2^n	n	2^n
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1,024 (1K)	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096 (4K)	20	1,048,576 (1M)
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

1.2 Binary Numbers (5/7)

- Arithmetic Operation
 - Arithmetic operations with numbers in base r follow the same rules as decimal numbers.
- **Practice Exercise 1.1** What is the decimal value of $(1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0)$?
Answer: 21

1.2 Binary Numbers (6/7)

- Binary Arithmetic
 - Single Bit Addition with Carry
 - Multiple Bit Addition
 - Single Bit Subtraction with Borrow
 - Multiple Bit Subtraction
 - Multiplication
 - BCD Addition

1.2 Binary Numbers (7/7)

● Binary Addition

$$\begin{array}{r}
 \text{被加數 Augend: } 101101 \\
 \text{加數 Addend: } +100111 \\
 \hline
 \text{Sum: } 1010100
 \end{array}$$

● Binary Multiplication

$$\begin{array}{r}
 \text{被乘數 Multiplicand: } 1011 \\
 \text{乘數 Multiplier: } \times 101 \\
 \hline
 \text{Partial Products: } 1011 \\
 0000- \\
 1011- \\
 \hline
 \text{Product: } 110111
 \end{array}$$

● Binary Subtraction

$$\begin{array}{r}
 \text{被減數 Minuend: } 101101 \\
 \text{減數 Subtrahend: } -100111 \\
 \hline
 \text{Difference: } 000110
 \end{array}$$

1.3 Number-Base Conversions (1/6)

Name	Radix	Digits
Binary	2	0, 1
Octal	8	0, 1, 2, 3, 4, 5, 6, 7,
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Hexadecimal	16	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

- The six letters (in addition to the 10 integers) in hexadecimal represent: 10, 11, 12, 13, 14, and 15, respectively.

1.3 Number-Base Conversions (2/6)

- **Example 1.1** Convert decimal 41 to binary. The process is continued until the integer quotient becomes 0.

	Quotient		Remainder	Coefficient
$41/2=$	20	+	1/2	$a_0 = 1$
$20/2=$	10	+	0	$a_1 = 0$
$10/2=$	5	+	0	$a_2 = 0$
$5/2=$	2	+	1/2	$a_3 = 1$
$2/2=$	1	+	0	$a_4 = 0$
$1/2=$	0	+	1/2	$a_5 = 1$

→ $(41)_{10} = (a_5 a_4 a_3 a_2 a_1 a_0)_2 = (101001)_2$

1.3 Number-Base Conversions (3/6)

- The arithmetic process can be manipulated more conveniently as follows:

Integer	Remainder
41	
20	1
10	0
5	0
2	1
1	0
0	1

The answer is $(101001)_2$.

1.3 Number-Base Conversions (4/6)

- **Example 1.2** Convert decimal 153 to octal. The required base r is 8.

Integer	Remainder
153	
19	1
2	3
0	2

The answer is $(231)_8$

1.3 Number-Base Conversions (5/6)

- **Example 1.3** Convert $(0.6875)_{10}$ to binary.

	Integer		Fraction	Coefficient
$0.6875 \times 2 =$	1	+	0.3750	$a_{-1} = 1$
$0.3750 \times 2 =$	0	+	0.7500	$a_{-2} = 0$
$0.7500 \times 2 =$	1	+	0.5000	$a_{-3} = 1$
$0.5000 \times 2 =$	1	+	0.0000	$a_{-4} = 1$



$$\begin{aligned}
 (0.6875)_{10} \\
 &= (0.a_{-1}a_{-2}a_{-3}a_{-4})_2 \\
 &= (0.1011)_2
 \end{aligned}$$

- The process is continued until the fraction becomes 0 or until the number of digits has sufficient accuracy.
- To convert a decimal fraction to a number expressed in base r , a similar procedure is used. However, multiplication is by r instead of 2, and the coefficients found from the integers may range in value from 0 to $r - 1$ instead of 0 and 1.

1.3 Number-Base Conversions (6/6)

- **Example 1.4** Convert $(0.513)_{10}$ to octal.

$$\begin{array}{l}
 0.513 \times 8 = 4.104 \\
 \leftarrow \\
 0.104 \times 8 = 0.832 \\
 \leftarrow \\
 0.832 \times 8 = 6.656 \\
 \leftarrow \\
 0.656 \times 8 = 5.248 \\
 \leftarrow \\
 0.248 \times 8 = 1.984 \\
 \leftarrow \\
 0.984 \times 8 = 7.872
 \end{array}
 \quad \Rightarrow \quad (0.513)_{10} = (0.406517\ldots)_8$$

- From Examples 1.1 and 1.3: $(41.6875)_{10} = (101001.1011)_2$
- From Examples 1.2 and 1.4: $(153.513)_{10} = (231.406517)_8$
- **Practice Exercise 1.2** Convert $(117.23)_{10}$ to octal.
Answer: $(117.23)_{10} = (165.1656)_8$

1.4 Octal and Hexadecimal Numbers (1/3)

- Numbers with different bases: Table 1.2.

Table 1.2
Numbers with Different Bases

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

1.4 Octal and Hexadecimal Numbers (2/3)

- Conversion from **binary to octal** can be done by positioning the binary number into groups of three digits each, starting from the binary point and proceeding to the left and to the right.

$$\begin{array}{ccccccc} (10 & 110 & 001 & 101 & 011 & \cdot & 111 & 100 & 000 & 110)_2 = (26153.7406)_8 \\ 2 & 6 & 1 & 5 & 3 & & 7 & 4 & 0 & 6 \end{array}$$

- Conversion from **binary to hexadecimal** is similar, except that the binary number is divided into groups of four digits:

$$\begin{array}{ccccccc} (10 & 1100 & 0110 & 1011 & \cdot & 1111 & 0010)_2 = (2C6B.F2)_{16} \\ 2 & C & 6 & B & & F & 2 \end{array}$$

- Conversion from **octal or hexadecimal to binary** is done by reversing the preceding procedure.

$$\begin{array}{ccccccc} (673.124)_8 = (110 & 111 & 011 & \cdot & 001 & 010 & 100)_2 & (306.D)_{16} = (0011 & 0000 & 0110 & \cdot & 1101)_2 \\ 6 & 7 & 3 & & 1 & 2 & 4 & 3 & 0 & 6 & & D \end{array}$$

1.4 Octal and Hexadecimal Numbers (3/3)

- **Practice Exercise 1.3** Find the binary representation of $(135)_{10}$.
Answer: $(135)_{10} = (10000111)_2$
- **Practice Exercise 1.4** Find the octal representation of $(135)_{10}$.
Answer: $(135)_{10} = (207)_8$

1.5 Complements of Numbers (1/9)

- There are two types of complements for each base- r system
 - The radix complement: r 's complement
 - The diminished radix complement: $(r - 1)$'s complement
- Diminished radix complement
 - Given a number N in base r having n digits, the $(r - 1)$'s complement of N is defined as $(r^n - 1) - N$.
 - For decimal numbers, $r = 10$ and $(r - 1) = 9$, so the 9's complement of N is $(10^n - 1) - N$.
- Examples
 - The 9's complement of a decimal 546700 is $999999 - 546700 = 453299$.
 - The 9's complement of a decimal 012398 is $999999 - 012398 = 987601$.

1.5 Complements of Numbers (2/9)

- For binary numbers, $r = 2$ and $r - 1 = 1$, so the 1's complement of N is $(2^n - 1) - N$.
- Examples
 - The 1's complement of 1011000 is 0100111.
 - The 1's complement of 0101101 is 1010010.

1.5 Complements of Numbers (3/9)

- Radix Complement (r 's complement)
 - The r 's complement of an n -digit number $(N)_r$ is defined as $r^n - N$ for $N \neq 0$ and as 0 for $N = 0$.
 - It can also be obtained by adding 1 to the $(r - 1)$'s complement since $r^n - N = [(r^n - 1) - N] + 1$.
- Example: Base-10
 - The 10's complement of 012398 is 987602
 - The 10's complement of 246700 is 753300
- Example: Base-2
 - The 2's complement of 1101100 is 0010100
 - The 2's complement of 0110111 is 1001001
- **Practice Exercise 1.5** Find (a) the diminished radix (9's) complement and (b) the radix (10's) complement of $(135)_{10}$.
Answer: (a) 9's complement: $(864)_{10}$; (b) 10's complement: $(865)_{10}$

1.5 Complements of Numbers (4/9)

- Subtraction with Complements: The subtraction of two n -digit unsigned numbers $M - N$ in base r can be done as follows:
 1. Adding the minuend M to the r 's complement of the subtrahend N .
Mathematically, $M + (r^n - N) = M - N + r^n$.
 2. If $M \geq N$, the sum will produce an end carry r^n , which can be discarded; what is left is the result $M - N$.
 3. If $M < N$, the sum does not produce an end carry and is equal to $r^n - (N - M)$, which is the r 's complement of $(N - M)$. To obtain the answer in a familiar form, take the r 's complement of the sum and place a negative sign in front.

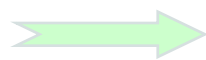
1.5 Complements of Numbers (5/9)

- **Example 1.5** Using 10's complement, subtract $72532 - 3250$.

$$\begin{array}{r}
 M = \quad 72532 \\
 10\text{'s complement of } N = \quad +96750 \\
 \hline
 \text{Sum} = \quad 169282 \\
 \text{Discard end carry } 10^5 \quad -100000 \\
 \hline
 \text{Answer} \quad 69282
 \end{array}$$

- **Example 1.6** Using 10's complement, subtract $3250 - 72532$

$$\begin{array}{r}
 M = \quad 03250 \\
 10\text{'s complement of } N = \quad +27468 \\
 \hline
 \text{Sum} = \quad 30718
 \end{array}
 \quad \longrightarrow \quad \text{There is no end carry.}$$

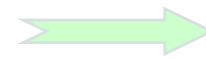

 Therefore, the answer is $-(10\text{'s complement of } 30718) = -69282$.

1.5 Complements of Numbers (6/9)

- **Example 1.7** Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$ and (b) $Y - X$ by using 2's complement.

$$\begin{array}{rcl}
 \text{(a)} & X = & 1010100 \\
 & 2\text{'s complement of } Y = & +0111101 \\
 & \text{Sum} = & \hline & 10010001 \\
 & \text{Discard end carry } 2^7 & -10000000 \\
 & \text{Answer} & \hline & 0010001
 \end{array}$$

$$\begin{array}{rcl}
 \text{(b)} & Y = & 1000011 \\
 & 2\text{'s complement of } X = & +0101100 \\
 & \text{Sum} = & \hline & 1101111
 \end{array}$$



There is no end carry. Therefore, the answer is $Y - X = - (2\text{'s complement of } 1101111) = -0010001$.

1.5 Complements of Numbers (7/9)

- Subtraction of unsigned numbers can also be done by means of the $(r - 1)$'s complement. Remember that the $(r - 1)$'s complement is one less than the r 's complement.
- Example 1.8** Repeat Example 1.7, but this time using 1's complement.

(a)

$$\begin{array}{r}
 X = \quad \quad \quad 1010100 \\
 1\text{'s complement of } Y = \quad +0111100 \\
 \hline
 \text{Sum} = \quad \quad \quad 10010000 \\
 \text{End-around carry} = \quad \quad \quad +1 \\
 \hline
 \text{Answer} \quad \quad \quad 0010001
 \end{array}$$

There is end carry, the answer of $X - Y = \text{Sum} + 1 = 0010001$.

(b)

$$\begin{array}{r}
 Y = \quad \quad \quad 1000011 \\
 1\text{'s complement of } X = \quad +0101100 \\
 \hline
 \text{Sum} = \quad \quad \quad 1101111
 \end{array}$$

There is no end carry, the answer of $Y - X = -(1\text{'s complement of } 1101110) = -0010001$.

1.5 Complements of Numbers (8/9)

- **Practice Exercise 1.6** Given $X = (1101010)_2$ and $Y = (0101011)_2$, (a) find $X - Y$ and (b) find $Y - X$ using 2's complements.

Answer:

- a) Since $X = (1101010)_2 = (106)_{10}$ and $Y = (0101011)_2 = (43)_{10}$
 $X - Y = (106)_{10} - (43)_{10} = (63)_{10}$
 2's complement of Y : $(1010101)_2$
 $X - Y = (1101010)_2 + (1010101)_2 = (0111111)_2 = (63)_{10}$
- b) For $Y - X = (43)_{10} - (106)_{10} = -(63)_{10} = (0101011)_2$
 2's complement of X : $(0010110)_2$
 $Y - X = (0101011)_2 + (0010110)_2 = (1000001)_2 \rightarrow$ no end carry
 $Y - X = 2$'s complement of $(1000001)_2$
 $Y - X = -(0111111)_2 = -(63)_{10}$

1.5 Complements of Numbers (9/9)

- **Practice Exercise 1.7** Repeat Practice Exercise 1.6 using 1's complements.

Answer:

- a) Since $X = (1101010)_2 = (106)_{10}$ and $Y = (0101011)_2 = (43)_{10}$
 $X - Y = (106)_{10} - (43)_{10} = (63)_{10}$
 1's complement of Y : $(1010100)_2$
 $X - Y = (1101010)_2 + (1010100)_2 = (10111110)_2 \rightarrow$ End-around carry
 $X - Y = (1101010)_2 + (1010100)_2 = (0111111)_2 = (63)_{10}$
- b) For $Y - X = (43)_{10} - (106)_{10} = -(63)_{10}$ $(0101011)_2$
 1's complement of X : $(0010101)_2$
 $Y - X = (0101011)_2 + (0010101)_2 = (1000000)_2 \rightarrow$ no end-around carry
 $Y - X = 1$'s complement of $(1000000)_2$
 $Y - X = -(0111111)_2 = -(63)_{10}$

1.6 Signed Binary Numbers (1/6)

- How to represent negative integers?
 - Represent the sign with a bit placed in the leftmost position of the number.
 - The sign bit 0 for positive and 1 for negative.
- Example:
 - Signed-magnitude representation: 10001001
 - Signed-1's-complement representation: 11110110
 - Signed-2's-complement representation: 11110111
- **Practice Exercise 1.8** Which bit of signed binary number represents the sign? **Answer:** The leftmost bit
- **Practice Exercise 1.9** What unsigned binary number is represented by the string of bits 11001? **Answer:** $(25)_{10}$

1.6 Signed Binary Numbers (2/6)

- **Practice Exercise 1.10** What decimal number does the signed-magnitude binary number $N = (10011)_2$ represent?

Answer: $N = -(3)_{10}$

- **Practice Exercise 1.11** Convert the signed-magnitude binary number $N = (01100)_2$ to a negative value having the same magnitude.

Answer: $N = (11100)_2$

- **Practice Exercise 1.12** Represent $-(5)_{10}$ three ways with 8 bits: (a) signed-magnitude (b) signed-1's-comp., and (c) signed-2's-comp..

Answer: (a) $(10000101)_2$, (b) $(11111010)_2$, and (c) $(11111011)_2$

- **Practice Exercise 1.13** In the signed-2's-comp. system, negate the number $(7)_{10}$, represented with 8 bits.

Answer: $(7)_{10} = (0000\ 0111)_2$, 1's comp. = $(1111\ 1000)_2$, 2's comp. = $(1111\ 1001)_2$.

1.6 Signed Binary Numbers (3/6)

Table 1.3
Signed Binary Numbers

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

1.6 Signed Binary Numbers (4/6)

- Arithmetic Addition:

- The addition of two signed binary numbers with negative numbers represented in signed-2's-complement form is obtained from the addition of the two numbers, including their sign bits.
- A carry out of the sign-bit position is **discarded**.

- Example:

+ 6	00000110	– 6	11111010
<u>+13</u>	<u>00001101</u>	<u>+13</u>	<u>00001101</u>
+19	00010011	+ 7	1 00000111
+ 6	00000110	– 6	11111010
<u>–13</u>	<u>11110011</u>	<u>–13</u>	<u>11110011</u>
–7	11111001	–19	1 11101101

1.6 Signed Binary Numbers (5/6)

- Arithmetic Subtraction (in 2's-complement form):
 1. Take the 2's complement of the subtrahend (including the sign bit) and add it to the minuend (including sign bit).
 2. A carry out of sign-bit position is **discarded**.

$$\begin{aligned} &\longrightarrow (\pm A) - (+B) = (\pm A) + (-B) \\ &\quad (\pm A) - (-B) = (\pm A) + (+B) \end{aligned}$$

- Example: $(-6)_{10} - (-13)_{10}$

$$\begin{aligned} &\longrightarrow (11111010)_2 - (11110011)_2 \\ &\longrightarrow = (11111010)_2 + (00001101)_2 \\ &\longrightarrow = 1(00000111)_2 = (+7)_{10} \end{aligned}$$

1.6 Signed Binary Numbers (6/6)

• Practice Exercise 1.14

$$\begin{array}{r} (a) \ +4 \\ +11 \end{array}$$

$$\begin{array}{r} (c) \ +4 \\ -11 \end{array}$$

$$\begin{array}{r} (b) \ -4 \\ +11 \end{array}$$

$$\begin{array}{r} (d) \ -4 \\ -11 \end{array}$$

Answer: Assume 8 bits are used to represent the signed binary numbers.

$$\begin{array}{rcl} (a) \ +4 & 0000 \ 0100 & (c) \ +4 \quad 0000 \ 0100 \\ +11 & \underline{0000 \ 1011} & -11 \quad \underline{1111 \ 0101} \\ +15 & 0000 \ 1111 & -7 \quad 1111 \ 1001 \end{array}$$

$$\begin{array}{rcl} (b) \ -4 & 1111 \ 1100 & (d) \ -4 \quad 1111 \ 1100 \\ +11 & \underline{0000 \ 1011} & -11 \quad \underline{1111 \ 0101} \\ +7 & 10000 \ 0111 & -15 \quad 11111 \ 0001 \end{array}$$

1.7 Binary Codes (1/8)

● Binary-Coded Decimal (BCD)

- A BCD number greater than 10 looks different from its equivalent binary number, even though both contain 1's and 0's.
- A number with k decimal digits will require $4k$ bits in BCD.
- A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9.
- The binary combinations 1010 through 1111 are not used and have no meaning in BCD.
- In general, when representing a decimal number in BCD, more number of bits are required than in binary.

Table 1.4
Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

- **Practice Exercise 1.15** Find the BCD representation of $(84)_{10}$.

Answer: $(84)_{10} = (1000\ 0100)_{\text{BCD}} = (1010100)_2$

1.7 Binary Codes (2/8)

- BCD Addition

$$\begin{array}{r} 4 \quad 0100 \\ +5 \quad 0101 \\ \hline 9 \quad 1001 \end{array}$$

$$\begin{array}{r} +4 \quad 0100 \\ +8 \quad +1000 \\ \hline 12 \quad 1100 \\ \quad +0110 \\ \hline 10010 \end{array}$$

$$\begin{array}{r} 8 \quad 1000 \\ +9 \quad +1001 \\ \hline 17 \quad 10001 \\ \quad +0110 \\ \hline 10111 \end{array}$$

- Example: Consider the addition of $184 + 576 = 760$ in BCD:

BCD		1	1	
	0001	1000	0100	184
	+0101	0111	0110	+576
Binary sum	0111	10000	1010	
Adding 6		0110	0110	
BCD sum	0111	0110	0000	760

1.7 Binary Codes (3/8)

- **Practice Exercise 1.16** Find the BCD sum of $(4)_{10} + (6)_{10}$.

Answer: $(0001\ 0000)_{\text{BCD}}$

- Using signed-10's-complement to represent a signed BCD.
 - The signed bits for “+” and “−” are $(0000)_{\text{BCD}}$ and $(1001)_{\text{BCD}}$, respectively.
 - For a negative BCD number, the magnitude bits are using the 10's complement.
 - Computing $(+375)_{10} + (-240)_{10}$, $(+375)_{10} \rightarrow (0375)_{\text{BCD}}$, $(-240)_{10} \rightarrow (9760)_{\text{BCD}}$

0 375

+9 760

10 135 The end carry is **discarded**.

- **Practice Exercise 1.17** Find the BCD sum (a) $370 + (-250)$; (b) $250 + (-370)$

Answer: (a) $(0120)_{\text{BCD}}$; (b) $(9880)_{\text{BCD}} = (-120)_{10}$

1.7 Binary Codes (4/8)

● Other decimal codes

Table 1.5 Some digits are not uniquely encoded.
Four Different Binary Codes for the Decimal Digits

Decimal Digit	BCD 8421	2421	Excess-3	Self-complementing code 8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
Unused	1010	0101	0000	0001
bit	1011	0110	0001	0010
combi-	1100	0111	0010	0011
nations	1101	1000	1101	1100
	1110	1001	1110	1101
	1111	1010	1111	1110

Table 1.6 Only one bit in the code group changes in going from one number to the next.
Gray Code

Gray Code	Decimal Equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

1.7 Binary Codes (5/8)

- American Standard Code for Information Interchange (ASCII)
- A popular code used to represent information sent as character-based data.
- It uses 7 bits ($b_7b_6b_5b_4b_3b_2b_1$) to represent:
 - 94 Graphic printing characters
 - 26 uppercase and 26 lowercase letters
 - 10 numbers and 32 special printable characters (e.g., %, *, and \$)
 - 34 Non-printing characters
 - Format effectors: BS (Backspace), CR (Carriage Return), HT (Horizontal Tabulation), ...
 - Information separators: RS (Record Separator), FS (File Separator), ...
 - Communication control: STX (Start of Text), ETX (End of Text), ...

1.7 Binary Codes (6/8)

Table 1.7
American Standard Code for Information Interchange (ASCII)

$b_4b_3b_2b_1$	$b_7b_6b_5$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	–	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	–	o	DEL

Control Characters

NUL	Null	DLE	Data-link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End-of-transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

1.7 Binary Codes (7/8)

- ASCII Properties
 - Digits 0 to 9 span hexadecimal values $(30)_{16}$ to $(39)_{16}$.
 - Upper case A – Z span $(41)_{16}$ to $(5A)_{16}$.
 - Lower case a – z span $(61)_{16}$ to $(7A)_{16}$.
 - **Lower to upper case translation (and vice versa) occurs by flipping bit 6.**
 - Delete (DEL) is all bits set, a carryover from when punched paper tape was used to store messages.
 - Punching all holes in a row erased a mistake!

1.7 Binary Codes (8/8)

- Error-Detecting Code
 - A *parity* bit is an extra bit included with a message to make the total number of 1's either even or odd.
 - Parity can only detect any **odd** combination errors.
- Example: The following two characters and their even and odd parity:

	With even parity	With odd parity
ASCII A = 1000001	0 1000001	1 1000001
ASCII T = 1010100	1 1010100	0 1010100

- **Practice Exercise 1.18** What is the even parity bit of “,” = 0101100?
Answer: 1

1.8 Binary Storage and Registers (1/3)

- A binary cell
 - A device possesses two stable state
 - Store one bit of information
 - Examples: flip-flop circuits, ferrite cores, capacitor
- A register
 - A group of n binary cells with 2^n possible states
 - The most common device used to hold data
 - AX in x86 CPU
- Register transfer
 - A transfer of the information stored in one register to another
 - One of the major operations in digital system

1.8 Binary Storage and Registers (2/3)

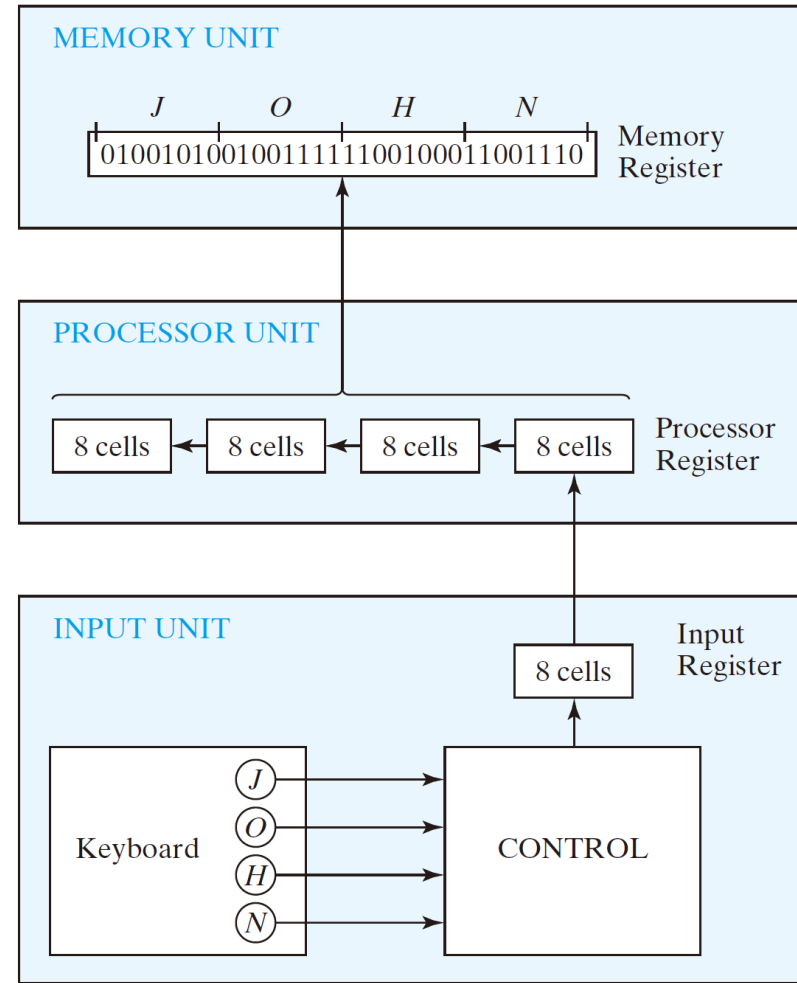


FIGURE 1.1
Transfer of information among registers

1.8 Binary Storage and Registers (3/3)

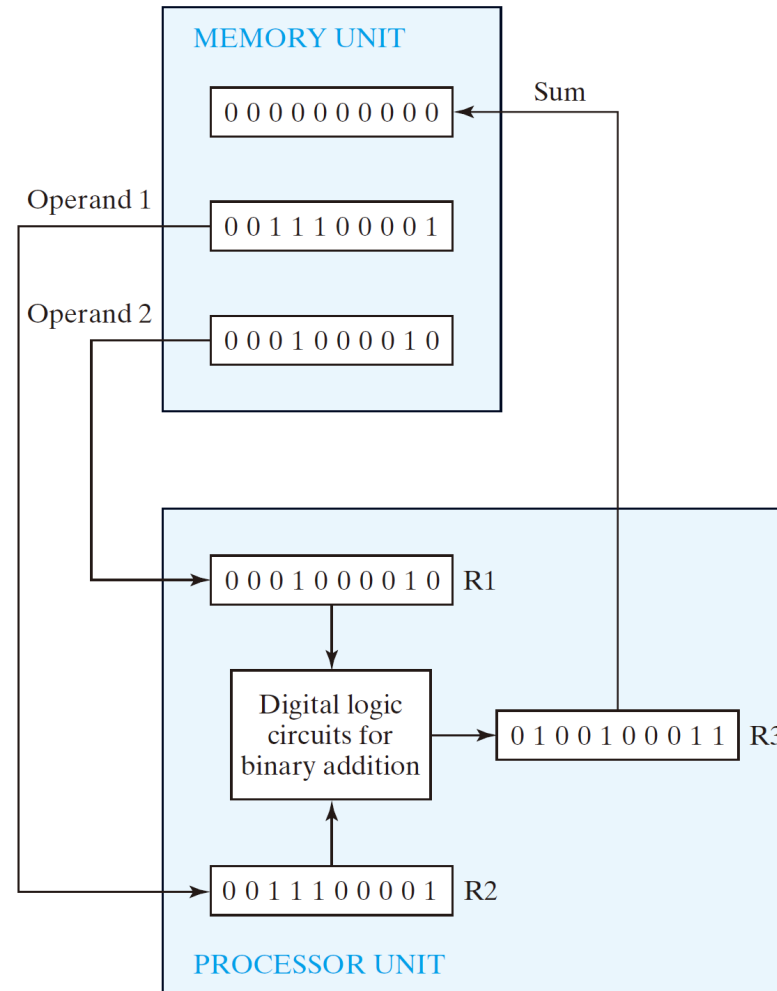


FIGURE 1.2
Example of binary information processing

1.9 Binary Logic (1/5)

- Definition of Binary Logic
 - Binary logic consists of binary variables and a set of logical operations.
 - The variables are designated by letters of the alphabet, such as A, B, C, x, y, z , etc., with each variable having two and only two distinct possible values: 1 and 0,
 - There are three basic logical operations: AND, OR, and NOT.
 - AND: $x \cdot y = z$ or $xy = z$. $\rightarrow z = 1$ if $x = 1$ and $y = 1$; otherwise $z = 0$.
 - OR: $x + y = z$. $\rightarrow z = 0$ if $x = 0$ and $y = 0$; otherwise $z = 1$.
 - NOT: $x' = z$ or $\bar{x} = z$. $\rightarrow$ if $x = 0$, then $z = 1$, but if $x = 1$, then $z = 0$. NOT is also known as the complement operation.

1.9 Binary Logic (2/5)

- The truth tables for AND, OR, and NOT are given in Table 1.8.

Table 1.8
Truth Tables of Logical Operations

AND			OR			NOT	
x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

1.9 Binary Logic (3/5)

- Example of binary signals

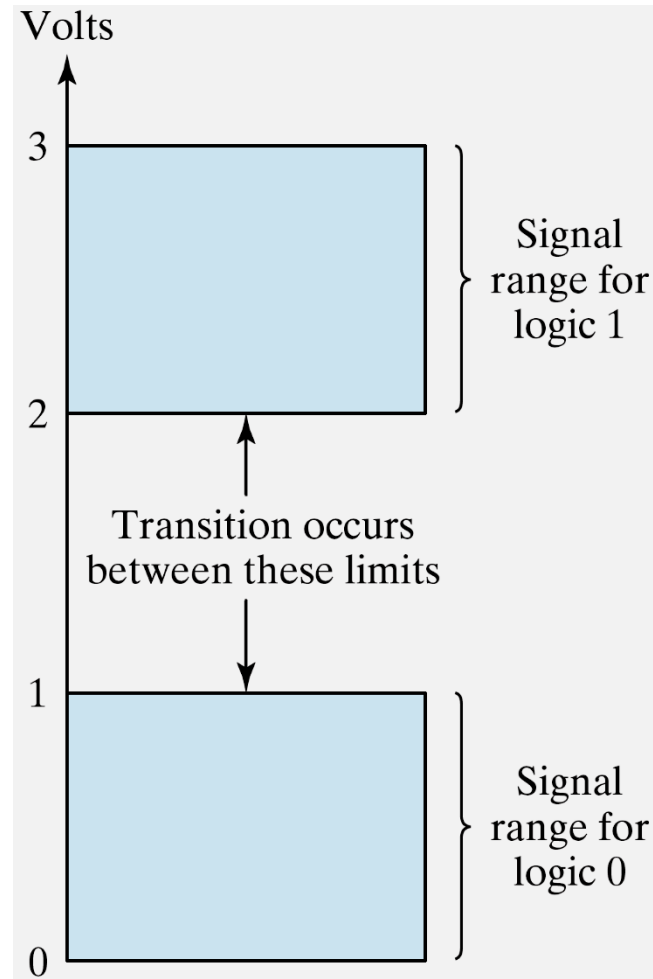


FIGURE 1.3 Signal levels for binary logic values

1.9 Binary Logic (4/5)

- Graphic symbols and input-output signals for logic gates:

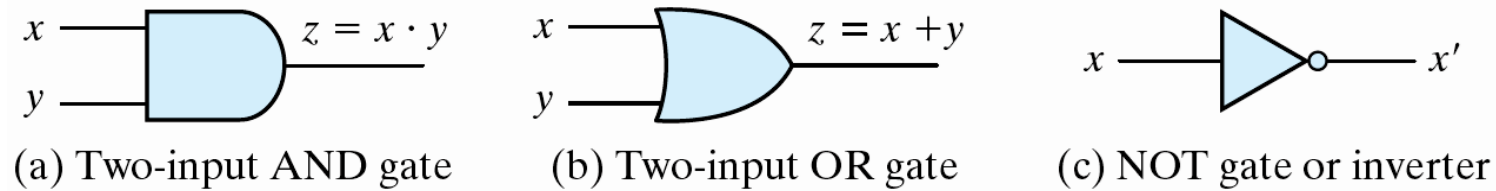


FIGURE 1.4 Symbols for digital logic circuits

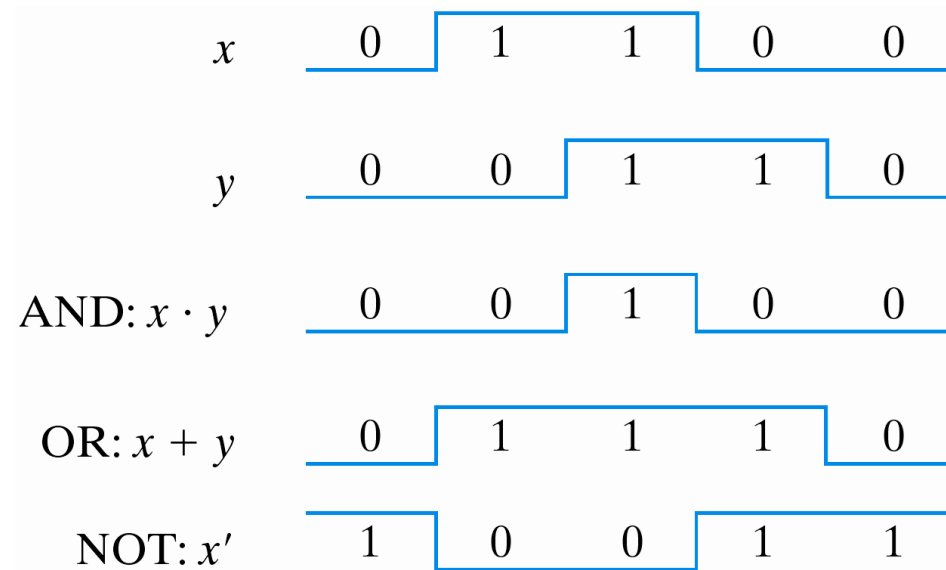


FIGURE 1.5 Input-Output signals for gates

1.9 Binary Logic (5/5)

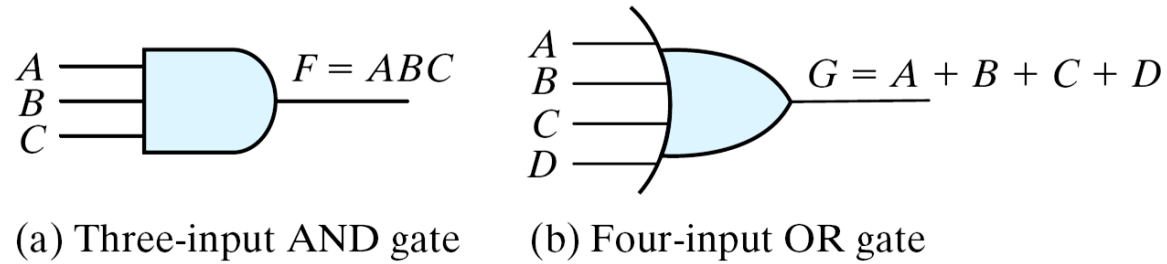


FIGURE 1.6 Gates with multiple inputs